Question 1 : Use Github Copilot to create a table SalesData whose columns are CustomerID, Name, Age, City, PurchaseAmount, PurchaseDate and then ask to insert 10,000 rows of random data.

- GitHub Copilot is an AI-powered coding assistant that helps developers generate SQL queries and code from natural-language instructions.
- **Step 1: Create the SalesData Table**
- Ask GitHub Copilot to generate the following SQL query:
- CREATE TABLE SalesData (
- CustomerID INT PRIMARY KEY,
- Name VARCHAR(100),
- Age INT,
- City VARCHAR(50),
- PurchaseAmount DECIMAL(10, 2),
- PurchaseDate DATE
-);
- This query creates a table named `SalesData` with six columns.
- **Step 2: Insert 10,000 Rows of Random Data**
- If you are using MySQL 8.0, you can ask GitHub Copilot to generate random data using a recursive CTE.
- SET SESSION cte_max_recursion_depth = 10000;
-
- INSERT INTO SalesData
- (CustomerID, Name, Age, City,
- PurchaseAmount, PurchaseDate)
- WITH RECURSIVE numbers AS (
- SELECT 1 AS n
- UNION ALL
- SELECT n + 1
- FROM numbers
- WHERE n < 10000
-)
- SELECT
- n,
- CONCAT('Customer_', n),
- FLOOR(18 + RAND() * 53),
- ELT(FLOOR(1 + RAND() * 5),
- 'Delhi', 'Mumbai', 'Kolkata',
- 'Chennai', 'Bengaluru'),
- ROUND(100 + RAND() * 9900, 2),
- DATE_ADD(
- '2024-01-01',
- INTERVAL FLOOR(RAND() * 1096) DAY
-)
- FROM numbers;
- This query generates 10,000 rows with randomized ages, cities, purchase amounts, and purchase dates. Customer IDs and names are unique.

- ## Step 3: Verify the Data
- Use the following queries to check the results.
- -- Display the first 10 records
- SELECT *
- FROM SalesData
- LIMIT 10;
-
- -- Count the total number of records
- SELECT COUNT(*) AS TotalRecords
- FROM SalesData;
- Expected output:
- `TotalRecords = 10000`

Question 2 : Use Copilot to: Find total sales per city Top 5 cities by revenue

GitHub Copilot can help generate SQL queries to analyze sales data and identify the cities that generate the highest revenue.

1. Find Total Sales per City

Ask GitHub Copilot: “Write an SQL query to calculate total sales for each city from the SalesData table.”

- SELECT
- City,
- SUM(PurchaseAmount) AS TotalSales
- FROM SalesData
- GROUP BY City
ORDER BY TotalSales DESC;

Explanation:

- `SUM(PurchaseAmount)` calculates total sales for each city.
- `GROUP BY City` groups records by city.
- `ORDER BY TotalSales DESC` sorts cities from highest to lowest sales.

2. Find the Top 5 Cities by Revenue

Ask GitHub Copilot: “Write an SQL query to find the top 5 cities with the highest total revenue.”

- SELECT
- City,
- SUM(PurchaseAmount) AS TotalRevenue
- FROM SalesData
- GROUP BY City

```
- ORDER BY TotalRevenue DESC
LIMIT 5;
```

Explanation:

- `SUM(PurchaseAmount)` calculates the revenue generated by each city.
- `GROUP BY City` combines sales records for the same city.
- `ORDER BY TotalRevenue DESC` ranks cities by revenue.
- `LIMIT 5` displays only the top five cities.

Question 3 : Ask Copilot: Find customers with purchases above average

GitHub Copilot can generate an SQL query to identify customers whose purchase amounts are greater than the average purchase amount of all records in the `SalesData` table.

SQL Query

Ask Copilot: “Write an SQL query to find customers whose purchases are above the average purchase amount.”

```
- SELECT
-     CustomerID,
-     Name,
-     City,
-     PurchaseAmount
- FROM SalesData
- WHERE PurchaseAmount > (
-     SELECT AVG(PurchaseAmount)
-     FROM SalesData
- )
ORDER BY PurchaseAmount DESC;
```

Explanation

- `SELECT` retrieves customer details and purchase amounts.
- `AVG(PurchaseAmount)` calculates the average purchase amount across all records.
- `WHERE` filters customers whose purchase amount is greater than the average.
- `ORDER BY PurchaseAmount DESC` displays the highest purchases first.

Question 4 : Identify and handle missing values in the dataset using Copilot.

- GitHub Copilot can help generate SQL queries to identify missing values in a dataset and handle them appropriately. Missing values may occur when customer details, city names, purchase amounts, or purchase dates are not available.
- ## Step 1: Identify Missing Values
- Ask GitHub Copilot: “Write an SQL query to identify missing values in each column of the SalesData table.”
- SELECT
- SUM(CASE WHEN CustomerID IS NULL THEN 1 ELSE 0 END) AS MissingCustomerID,
- SUM(CASE WHEN Name IS NULL OR TRIM(Name) = '' THEN 1 ELSE 0 END) AS MissingName,
- SUM(CASE WHEN Age IS NULL THEN 1 ELSE 0 END) AS MissingAge,
- SUM(CASE WHEN City IS NULL OR TRIM(City) = '' THEN 1 ELSE 0 END) AS MissingCity,
- SUM(CASE WHEN PurchaseAmount IS NULL THEN 1 ELSE 0 END) AS MissingPurchaseAmount,
- SUM(CASE WHEN PurchaseDate IS NULL THEN 1 ELSE 0 END) AS MissingPurchaseDate
- FROM SalesData;
- Explanation: This query counts missing values in each column. It also treats empty or whitespace-only names and cities as missing.
- ## Step 2: Display Rows Containing Missing Values
- SELECT *
- FROM SalesData
- WHERE CustomerID IS NULL
- OR Name IS NULL OR TRIM(Name) = ''
- OR Age IS NULL
- OR City IS NULL OR TRIM(City) = ''
- OR PurchaseAmount IS NULL
- OR PurchaseDate IS NULL;
- This query displays all rows containing at least one missing value.
- ## Step 3: Handle Missing Values
- Missing values should be handled according to the business requirements.
- Method 1: Replace missing ages with the average age (MySQL 8.0)
- UPDATE SalesData
- SET Age = (
- SELECT avg_age
- FROM (
- SELECT AVG(Age) AS avg_age
- FROM SalesData
-) AS temp
-)
- WHERE Age IS NULL;
- Method 2: Replace missing city names with 'Unknown'
- UPDATE SalesData
- SET City = 'Unknown'

- WHERE City IS NULL OR TRIM(City) = '';
- Method 3: Identify records with missing purchase amounts
- SELECT *
- FROM SalesData
- WHERE PurchaseAmount IS NULL;
- Purchase amounts should not automatically be replaced with zero because that could distort sales analysis. Investigate the source data or exclude unresolved records from relevant calculations.

Question 5 : Task - Identify duplicate rows Remove duplicates using Copilot suggestion

GitHub Copilot can help generate SQL queries to identify duplicate records and remove them from a dataset. Duplicate records can lead to incorrect sales totals and misleading business reports.

Step 1: Identify Duplicate Rows

Ask GitHub Copilot: “Write an SQL query to identify duplicate rows in the SalesData table based on all columns except CustomerID.”

- SELECT
- Name,
- Age,
- City,
- PurchaseAmount,
- PurchaseDate,
- COUNT(*) AS DuplicateCount
- FROM SalesData
- GROUP BY
- Name,
- Age,
- City,
- PurchaseAmount,
- PurchaseDate
HAVING COUNT(*) > 1;

Explanation:

- `GROUP BY` groups records with identical values in the selected columns.
- `COUNT(*)` counts how many times each combination occurs.
- `HAVING COUNT(*) > 1` identifies combinations appearing more than once.

Step 2: Remove Duplicate Rows

In MySQL, if duplicate records should be identified by the five columns above and the record with the smallest `CustomerID` should be retained, use:

```
- DELETE s1
- FROM SalesData AS s1
- JOIN SalesData AS s2
-   ON s1.Name <=> s2.Name
-  AND s1.Age <=> s2.Age
-  AND s1.City <=> s2.City
-  AND s1.PurchaseAmount <=> s2.PurchaseAmount
-  AND s1.PurchaseDate <=> s2.PurchaseDate
 AND s1.CustomerID > s2.CustomerID;
```

Explanation:

- The self-join compares records within the same table.
- `<=>` compares values safely even when some columns contain `NULL`.
- `s1.CustomerID > s2.CustomerID` deletes the record with the larger ID and retains the smallest ID for each duplicate group.

Important: This query removes records that match on the five selected columns, even if their `CustomerID` values differ. Review the results and back up the table before deleting data.

Step 3: Verify Duplicate Removal

Run the identification query again:

```
- SELECT
-     Name,
-     Age,
-     City,
-     PurchaseAmount,
-     PurchaseDate,
-     COUNT(*) AS DuplicateCount
- FROM SalesData
- GROUP BY
-     Name,
-     Age,
-     City,
-     PurchaseAmount,
-     PurchaseDate
HAVING COUNT(*) > 1;
```

If no rows are returned, no duplicates remain under the selected criteria.

Question 6 : Generate: Column chart OR bar chart and Interpret which region has highest sales

GitHub Copilot can help generate SQL queries and charting code to visualize sales by region. A column chart or bar chart makes it easier to compare sales performance across different regions.

Step 1: Calculate Total Sales by Region

Ask Copilot: “Write an SQL query to calculate total sales for each region.”

Assuming the table contains a `Region` column:

```
- SELECT
-     Region,
-     SUM(PurchaseAmount) AS TotalSales
- FROM SalesData
- GROUP BY Region
ORDER BY TotalSales DESC;
```

Explanation:

- `SUM(PurchaseAmount)` calculates total sales.
- `GROUP BY Region` groups sales by region.
- `ORDER BY TotalSales DESC` sorts regions from highest to lowest sales.

Note: If your `SalesData` table does not contain a `Region` column, you will need a region field or a customer-to-region mapping before performing this analysis.

Step 2: Generate a Bar Chart Using Python

Ask Copilot: “Generate a bar chart using Matplotlib to display total sales by region and highlight the region with the highest sales.”

```
import pandas as pdimport matplotlib.pyplot as plt# Example sales datadata = {
    "Region": ["North", "South", "East", "West"],    "TotalSales": [250000, 180000, 220000, 300000]}df = pd.DataFrame(data)# Sort sales in descending orderdf = df.sort_values("TotalSales", ascending=False)# Generate bar chartplt.bar(df["Region"], df["TotalSales"])plt.title("Total Sales by Region")plt.xlabel("Region")plt.ylabel("Total Sales")plt.tight_layout()plt.show()# Identify the highest-sales regionhighest_region = df.loc[df["TotalSales"].idxmax()]print("Region with highest sales:", highest_region["Region"])print("Total sales:", highest_region["TotalSales"])
```

Step 3: Interpret the Chart

The example data produces the following result:

Total Sales by Region (Example Data)

Illustrative figures only, not actual SalesData results.

₹0₹80K₹160K₹240K₹320KNorthSouthEastWest

- West: Highest sales of ₹300,000.
- North: Sales of ₹250,000.
- East: Sales of ₹220,000.
- South: Lowest sales of ₹180,000.

These figures are illustrative; the actual highest-sales region can only be determined using your real dataset.

Question 7 : Create a pivot table showing total sales by region and product.

A Pivot Table is a data analysis tool used to summarize and organize large datasets. It helps compare total sales across different regions and products.

Step 1: Prepare the Dataset

Ensure your dataset contains these columns:

- Region
- Product
- Sales (or PurchaseAmount)

Step 2: Create a Pivot Table Using Excel

1. Select the entire dataset.
2. Go to Insert → PivotTable.
3. Choose the table or range and select where to place the PivotTable.
4. In the PivotTable Fields pane, drag:
    - Region → Rows
    - Product → Columns
    - Sales → Values
5. Ensure the Values field is set to Sum of Sales, not Count.

Step 3: Example Pivot Table

The following is an illustrative result:

Region	Laptop	Mobile	Tablet	Grand Total

East	50,000	30,000	20,000	100,000
North	70,000	40,000	25,000	135,000
South	60,000	35,000	15,000	110,000
West	80,000	50,000	30,000	160,000
Grand Total	260,000	155,000	90,000	505,000

Note: These values are examples, not actual results from your dataset.

Step 4: Use GitHub Copilot

You can ask Copilot:

> Create a PivotTable from my sales dataset showing Region in rows, Product in columns, and the sum of Sales in values. Include grand totals for rows and columns.

If you are working in SQL instead of Excel, the equivalent query is:

```
- SELECT
-     Region,
-     Product,
-     SUM(Sales) AS TotalSales
- FROM SalesData
- GROUP BY Region, Product
ORDER BY Region, TotalSales DESC;
```

Note: This SQL query assumes your table contains `Region`, `Product`, and `Sales` columns. Add these fields or use the corresponding column names in your actual dataset.

Question 8 : Use Copilot to generate a new calculated column Total Sales

- A calculated column is a new column created by applying a formula to existing columns in a dataset. GitHub Copilot can help generate the formula based on the requirements.

- ## Step 1: Ask GitHub Copilot
- Prompt:
- "Create a new calculated column named `Total Sales` that calculates the total sales amount for each record using Quantity and UnitPrice."
- ## Step 2: Create the Calculated Column
- If you are using Power BI, select Modeling → New column and enter this DAX formula:
- Total Sales = SalesData[Quantity] * SalesData[UnitPrice]
- This formula multiplies the quantity sold by the unit price for each row.
- ## Step 3: Example

Product	Quantity	UnitPrice	Total Sales
Laptop	2	40,000	80,000
Mobile	3	15,000	45,000
Tablet	4	10,000	40,000

- For example, the total sales for two laptops is:
- Total Sales = 2 × 40,000 = 80,000
- Note: If your dataset contains only a `PurchaseAmount` column, you can create a calculated column using:
- Total Sales = SalesData[PurchaseAmount]

Question 9: Summarize the dataset and find the key insights.

GitHub Copilot can help summarize a dataset by generating SQL queries, calculating key statistics, and identifying trends and patterns. These insights help businesses make better decisions.

Step 1: Summarize the Dataset

Ask GitHub Copilot:

"Write an SQL query to summarize the SalesData dataset by calculating total customers, average age, total sales, average purchase amount, and total number of records."

- SELECT

```
-       COUNT(*) AS TotalRecords,
-       COUNT(DISTINCT CustomerID) AS TotalCustomers,
-       ROUND(AVG(Age), 2) AS AverageAge,
-       ROUND(SUM(PurchaseAmount), 2) AS TotalSales,
-       ROUND(AVG(PurchaseAmount), 2) AS AveragePurchase
FROM SalesData;
```

Step 2: Find Key Insights

Use the following query to identify the highest-revenue cities:

```
-   SELECT
-       City,
-       SUM(PurchaseAmount) AS TotalRevenue
-   FROM SalesData
-   GROUP BY City
-   ORDER BY TotalRevenue DESC
LIMIT 5;
```

To find the highest individual purchases:

```
-   SELECT
-       CustomerID,
-       Name,
-       City,
-       PurchaseAmount
-   FROM SalesData
-   ORDER BY PurchaseAmount DESC
LIMIT 5;
```

Step 3: Interpret the Results

The analysis can reveal the following insights:

1. Overall sales performance: Total sales show the overall revenue generated by the dataset.
2. Customer spending: Average purchase amount indicates how much customers spend per transaction.
3. Top-performing cities: City-wise revenue identifies locations generating the most sales.
4. High-value customers: Customers with the largest purchases may be suitable for loyalty programs or targeted offers.
5. Data quality: Missing values and duplicate records can affect the accuracy of the results.

Question 10 : Generate VBA code using Copilot for cleaning data (removing duplicates, filling null values, format table)

GitHub Copilot can generate VBA (Visual Basic for Applications) code to automate data-cleaning tasks in Excel. The code below removes duplicate records, fills blank cells, and formats the dataset as a table.

Step 1: Ask GitHub Copilot

Use this prompt:

“Generate VBA code to clean my Excel dataset by removing duplicate rows, filling blank cells with `Unknown` for text columns and `0` for numeric columns, and formatting the data as a professional Excel table with headers.”

Step 2: VBA Code for Data Cleaning

The following example assumes your data is on a worksheet named `SalesData`, with headers in row 1.

```
-
-   Sub CleanSalesData()
-
-       Dim ws As Worksheet
-       Dim lastRow As Long
-       Dim lastCol As Long
-       Dim dataRange As Range
-       Dim i As Long
-       Dim cell As Range
-       Dim tbl As ListObject
-
-       ' Select worksheet
-       Set ws = ThisWorkbook.Worksheets("SalesData")
-
-       ' Find the last used row and column
-       lastRow = ws.Cells.Find(What:="*", _
-           SearchOrder:=xlByRows, _
-           SearchDirection:=xlPrevious).Row
-
-       lastCol = ws.Cells.Find(What:="*", _
-           SearchOrder:=xlByColumns, _
-           SearchDirection:=xlPrevious).Column
-
-       Set dataRange = ws.Range( _
-           ws.Cells(1, 1), ws.Cells(lastRow, lastCol))
-
-       ' Remove duplicate rows using all columns
-       dataRange.RemoveDuplicates _
-           Columns:=Evaluate("ROW(1:" & lastCol & ")"), _
-           Header:=xlYes
```

```
-
-       ' Recalculate last row after removing duplicates
-       lastRow = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
-       Set dataRange = ws.Range( _
-          ws.Cells(1, 1), ws.Cells(lastRow, lastCol))
-
-       ' Fill blank cells
-       For Each cell In dataRange.Offset(1, 0) _
-          .Resize(dataRange.Rows.Count - 1).Cells
-
-          If IsEmpty(cell.Value) Or Trim(CStr(cell.Value)) = "" Then
-
-             If IsNumeric(ws.Cells(1, cell.Column).Value) _
-                And InStr(1, ws.Cells(1, cell.Column).Value, _
-                "ID", vbTextCompare) = 0 Then
-
-                cell.Value = 0
-             Else
-                cell.Value = "Unknown"
-             End If
-
-          End If
-       Next cell
-
-       ' Format data as an Excel table
-       On Error Resume Next
-       Set tbl = ws.ListObjects("SalesTable")
-       On Error GoTo 0
-
-       If tbl Is Nothing Then
-          Set tbl = ws.ListObjects.Add( _
-             SourceType:=xlSrcRange, _
-             Source:=dataRange, _
-             XlListObjectHasHeaders:=xlYes)
-
-          tbl.Name = "SalesTable"
-       Else
-          tbl.Resize dataRange
-       End If
-
-       tbl.TableStyle = "TableStyleMedium2"
-       ws.Columns.AutoFit
-
-       MsgBox "Data cleaning completed successfully!", vbInformation
-
-   End Sub
```

Step 3: How to Run the VBA Code

1. Open your Excel workbook and save a backup.
2. Press `Alt + F11` to open the VBA Editor.
3. Select Insert → Module.
4. Paste the VBA code into the module.
5. Ensure your worksheet is named `SalesData`.
6. Run `CleanSalesData` by pressing `F5`.

Step 4: Explanation of the Code

Operation	VBA method	Purpose
Remove duplicates	`RemoveDuplicates`	Removes rows identical across all columns.
Fill blank cells	`IsEmpty` and `Trim`	Replaces blanks with `0` or `Unknown`.
Format as a table	`ListObjects.Add`	Creates a structured Excel table.
Apply table style	`TableStyleMedium2`	Improves readability.
Adjust column widths	`AutoFit`	Makes cell contents easier to read.

Important considerations:

- Filling missing values with `0` or `Unknown` is only an example. Choose replacements that make sense for each column; zero can distort sales analysis.
- The code removes exact duplicate rows across all columns, including the header being excluded from comparison.
- Run the macro on a backup first. It modifies the worksheet directly and assumes a non-empty dataset with headers in row 1.

-